AI Advances · Follow publication

Member-only story

RAG, LLM Wiki, or Gbrain? How Your Agent Remembers Changes Everything

Karpathy's compounding wiki, Garry Tan's autonomous brain, and the decision framework most teams skip

15 min read · Apr 27, 2026

Yanli Liu Follow

Listen Share More

Photo by Elisa on Unsplash

Read this for free

Three weeks ago, Andrej Karpathy posted a GitHub gist that hit 5,000 stars in days. His argument: stop using LLMs as search engines over your documents. Use them as knowledge engineers who compile, cross-reference, and maintain a living wiki. The system learns. The knowledge compounds.

Later on, Garry Tan shipped GBrain –24 autonomous skills, 21 cron jobs, and a brain spanning 17,888 pages. His system doesn't just remember things. It acts on them. Autonomously.

Both agree on the diagnosis: **RAG alone isn't enough**. Your agent re-reads the same documents for every question, never learning, never compounding, never connecting the dots between yesterday's insight and today's query. **It's a retriever, not a thinker.**

But their fixes take opposite directions.

> *Karpathy wants your agent to build a persistent, interlinked wiki — knowledge that grows richer with every source you feed it.*

> *Garry Tan wants your agent to operationalize that knowledge — skills that don't just know things but do things, running in the background while you sleep.*

And meanwhile, most production teams are still running vanilla **RAG.** Not because it's ideal, but because **it works at scale** and the alternatives are young.

So which architecture actually fits your agent? The answer depends on a question most "RAG is dead" articles never ask:

- what is your agent's job?
- Is it retrieving answers from a large corpus?
- Compiling knowledge that should grow over time?
- Or acting autonomously on what it knows?

Three patterns. Three trade-offs. One decision framework.

Why Agents Forget

Here's what most agent tutorials skip: the context window is not memory. It's a whiteboard that gets erased after every session.

Your agent can hold a million tokens in context. That sounds like a lot until you realize that degradation starts around 300,000–400,000 tokens — roughly 30–40% of the ceiling. And when the session ends, everything disappears. Your agent starts the next conversation from zero.

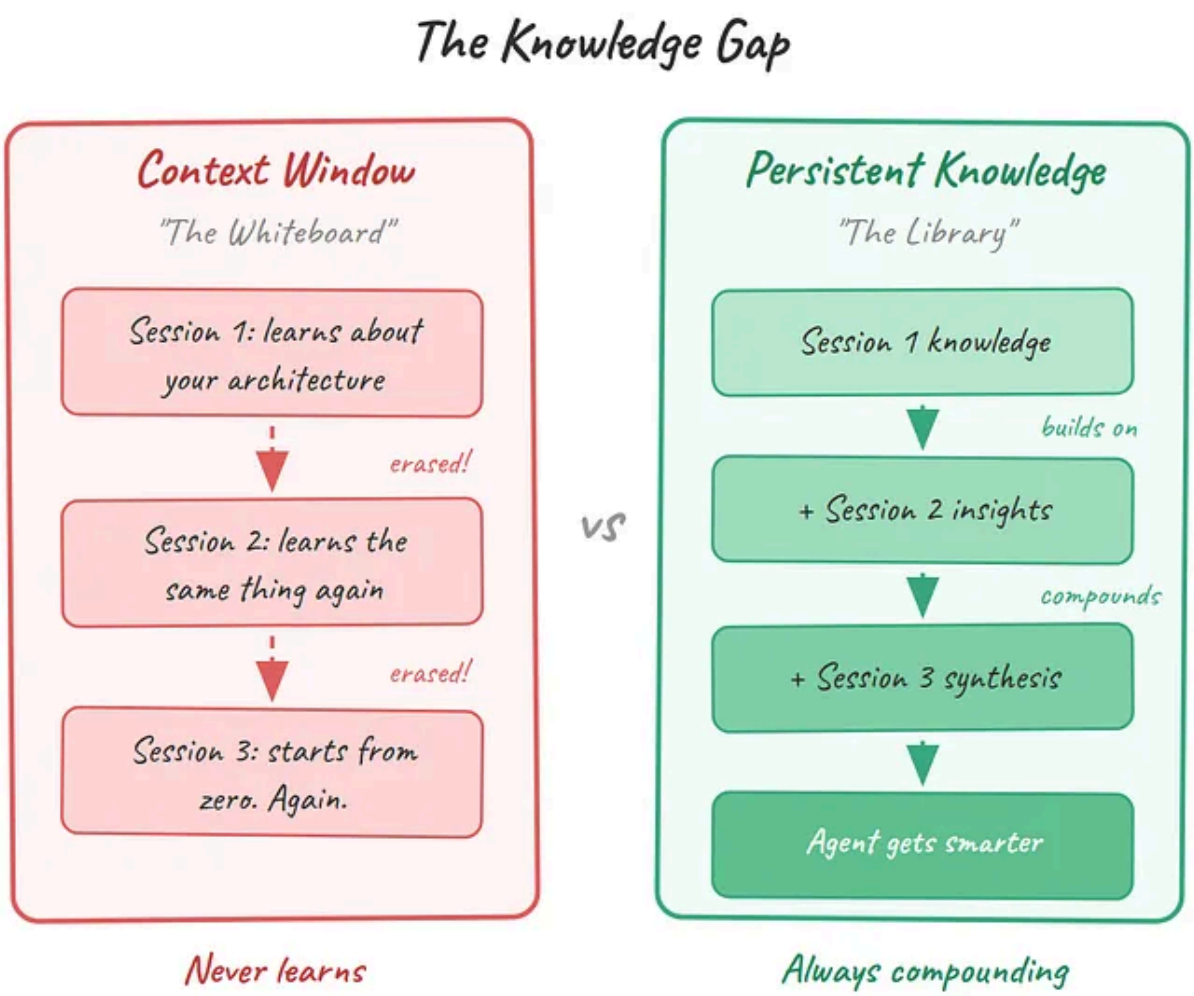

Diagram by Author: The Knowledge Gap — context window vs. persistent knowledge

RAG was the first serious answer to this problem. Instead of stuffing everything into the context window, you embed your documents into vectors, store them in a database, and retrieve relevant chunks at query time. It works. Millions of production systems run on it.

But RAG has a fundamental limitation that a 2024 research paper mapped into seven distinct failure points. Three of them matter most for agents:

The chunking problem. Your 30-page technical spec gets split into 500-token fragments. The chunk that mentions a compliance requirement lands in one vector. The chunk that explains why that requirement exists lands in another. The retriever finds one and misses the other. Your agent gives a technically correct but dangerously incomplete answer.

The re-derivation problem. Every query starts from scratch. Your agent analyzed the same architecture doc yesterday and drew the same conclusions. It'll do it again

tomorrow. RAG retrieves — it never learns. Karpathy put it sharply: “RAG rereads the same books for every exam, never actually learning the material.”

The passivity problem. RAG waits for you to ask. It never notices that the document it indexed last Tuesday contradicts the one it indexed today. It never flags that three sources disagree on a critical detail. It never acts on what it knows.

These three gaps — fragmented context, no compounding, no action — are precisely what the two new architectures target. Karpathy’s LLM Wiki attacks the first two. Garry Tan’s GBrain attacks all three.

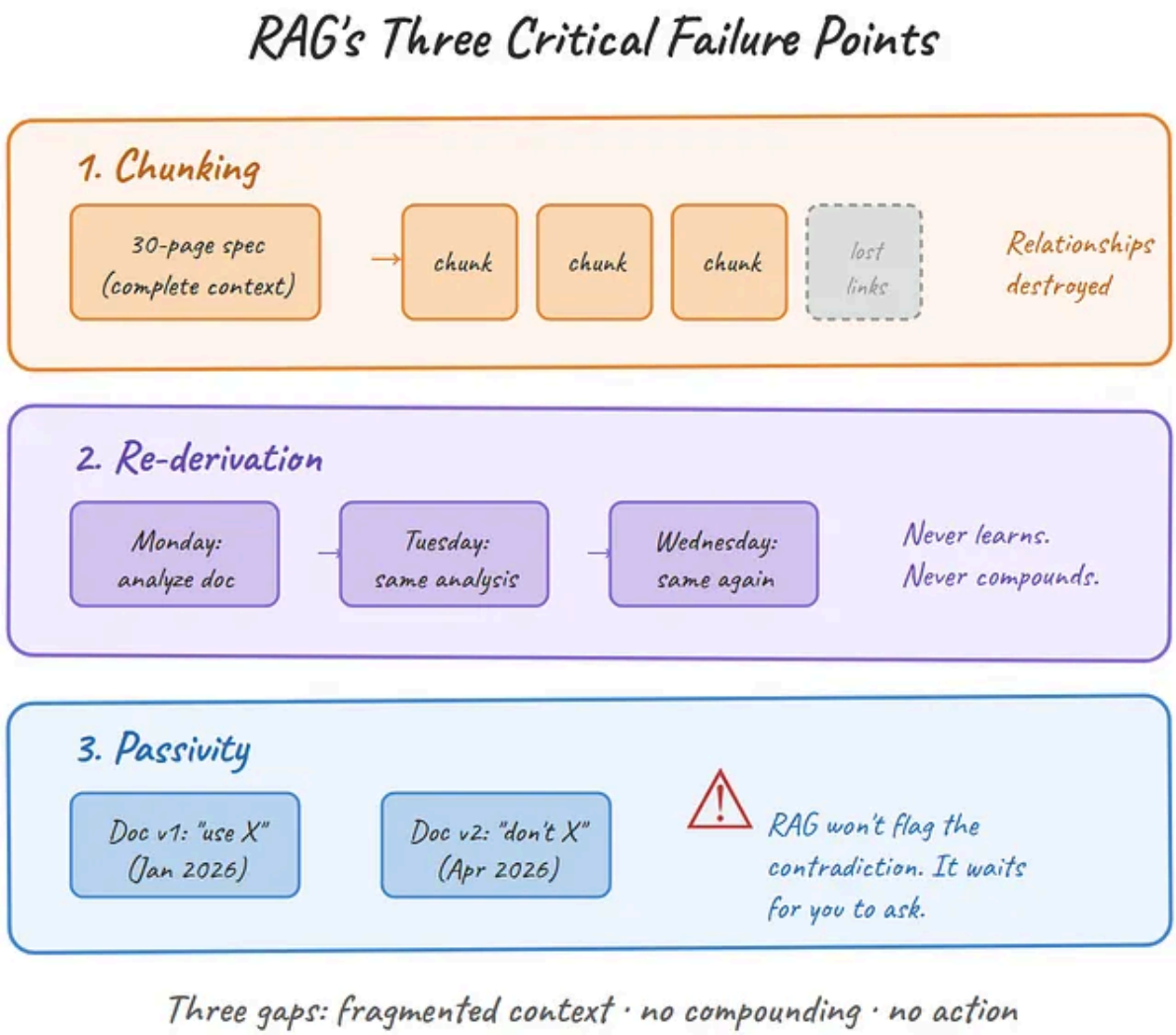

Diagram by Author: RAG’s Three Critical Failure Points

Architecture 1: RAG — The Retriever

RAG wins at scale and loses at depth. If you have 100,000 documents changing daily and need answers now, nothing else comes close. But your agent will never get smarter from using it.

How it works. You embed your documents into vectors, store them in a database like Pinecone or Chroma, and when a query arrives, you find the nearest chunks, inject them into the prompt, and let the model generate an answer. The pipeline is embed → store → retrieve → generate.

The architecture is mature. LangChain, LlamaIndex, and a dozen other frameworks have standardized it. Your team probably already knows how to build one. That matters more than most architectural comparisons admit.

Where it holds up. RAG handles corpus sizes that would choke the alternatives. A company with 200,000 internal documents — policies, memos, specs, Slack exports — can index everything and start answering questions the same day. No preprocessing into wiki pages. No engineering skills into markdown. Just embed and go.

It also handles freshness. When a document changes, you re-embed it. The next query gets the updated version. No wiki audit needed, no skill rewrite required.

Where it breaks. The chunking problem from Section 2 is structural, not fixable with better chunk sizes. A 2024 study identified seven failure points in production RAG systems — and three of the seven happen before the language model even sees the context.

There's also the cost that nobody talks about: latency. A RAG pipeline has multiple stages — embedding, vector search, reranking, context packaging — each adding milliseconds. For a single query, it's fine. For an agent making 40 tool calls in a loop, those milliseconds compound into seconds.

The enterprise reality. RAG is what most production teams run today because it's the architecture with the most battle scars. It has known failure modes, documented fixes, and an ecosystem of vendors competing to solve its problems. If you need to ship an internal knowledge assistant by next quarter, RAG is the proven path.

It's also the architecture compliance teams understand. Your data stays in your vector store. The retrieval is auditable. The generation is traceable. For regulated

industries, that auditability often outweighs the compounding benefits of newer approaches.

Verdict: Use RAG when your corpus is large (10,000+ documents), changes frequently, and your priority is shipping a production system with known trade-offs. Don't use it when your agent needs to learn from its own work or act autonomously.

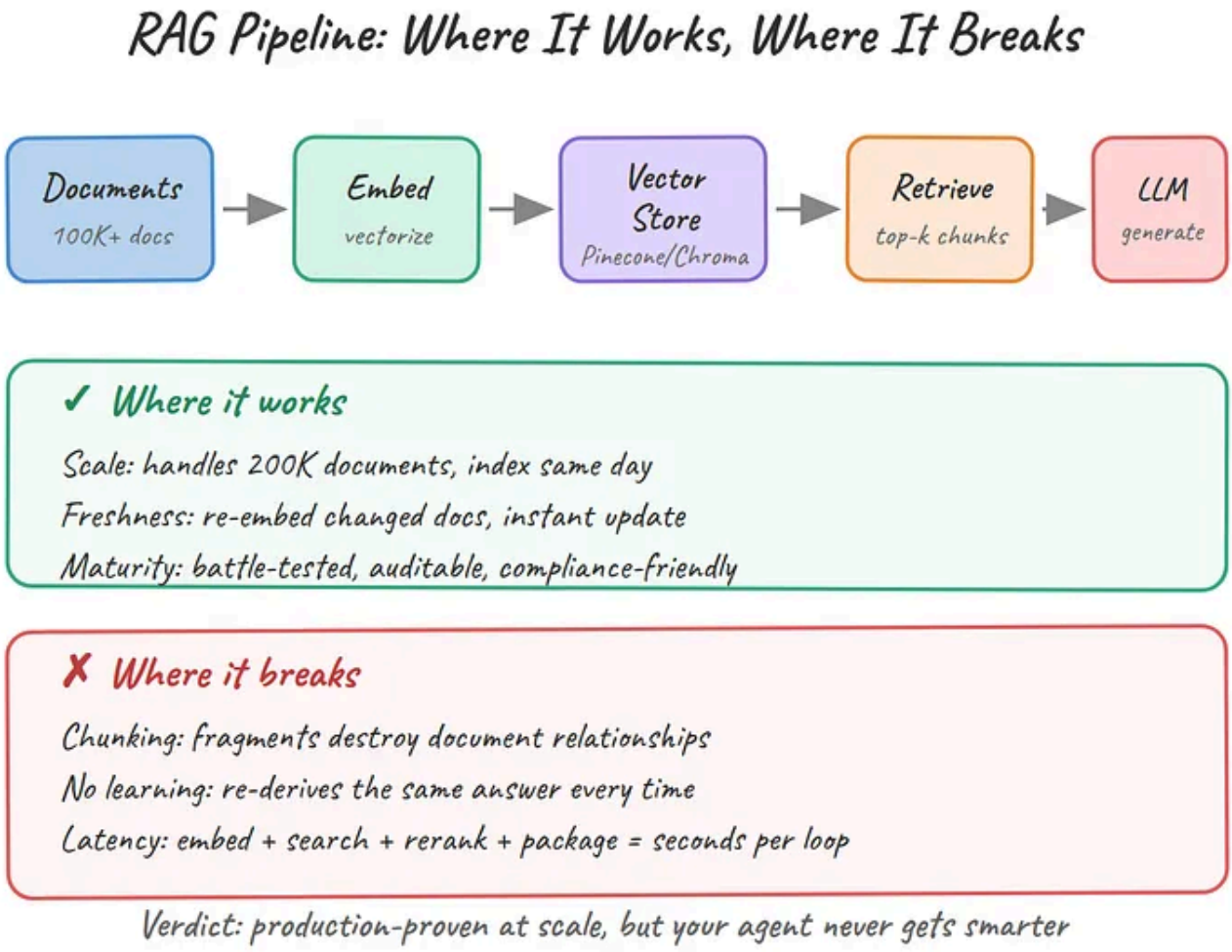

Diagram by Author: RAG architecture pipeline with failure points

Architecture 2: LLM Wiki — The Compiler

The LLM Wiki wins at depth and loses at scale. If you have fewer than 1,000 sources

Karpathy's LLM Wiki gist proposes a simple but powerful shift: instead of retrieving raw chunks at query time, use the LLM to pre-compile your sources into a

persistent, interlinked wiki. The model does the synthesis once. Every future query benefits from that work.

The three-layer architecture. Raw sources sit at the bottom — PDFs, articles, transcripts, bookmarks. These are immutable. The LLM reads them but never modifies them.

Above that sits the wiki itself — LLM-generated markdown pages containing summaries, entity pages, concept definitions, and cross-references. The LLM owns this layer entirely.

At the top sits the schema — a configuration file (think CLAUDE.md) that tells the LLM how to maintain the wiki: naming conventions, cross-referencing rules, what counts as a contradiction.

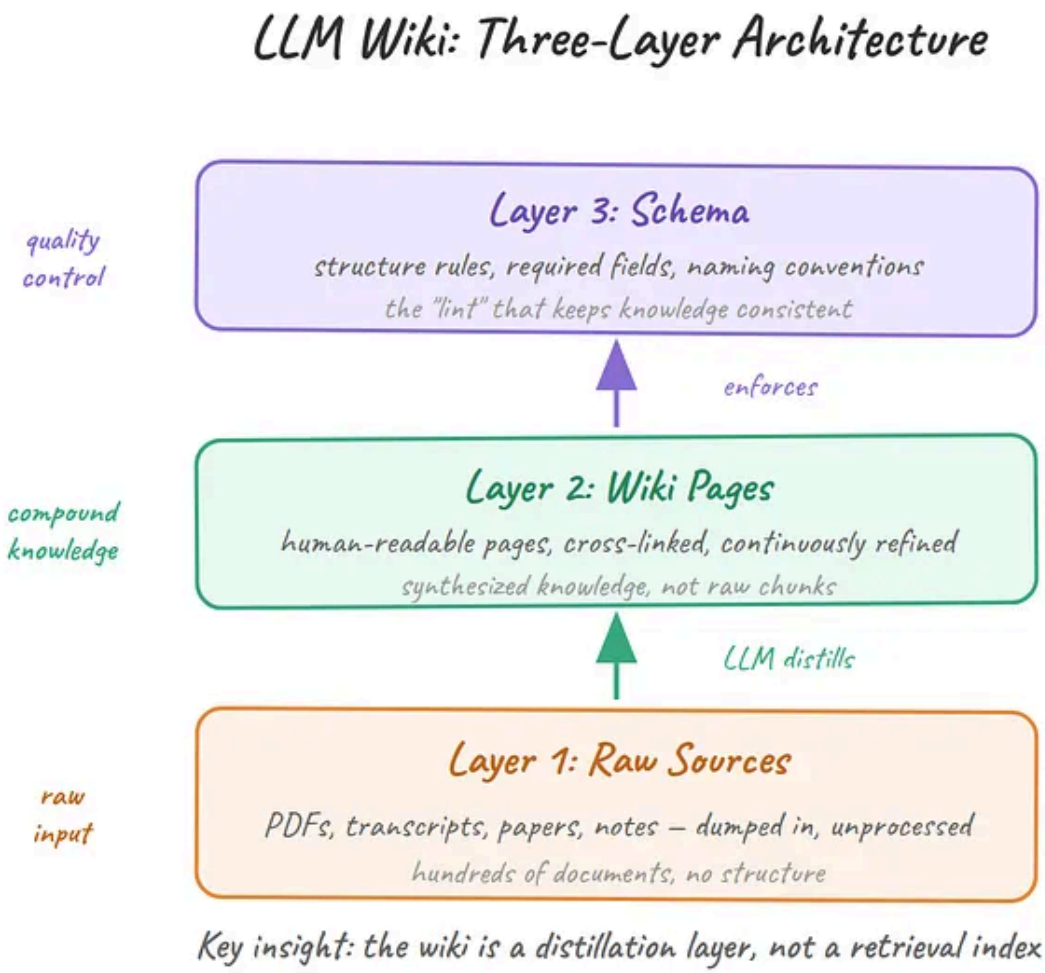

Diagram by Author: LLM Wiki three-layer architecture

How compounding works. When you feed the system a new document, the LLM doesn't just create a summary page. It reads the new source against the existing wiki

and updates every page that's affected. A single ingest typically touches 10–15 wiki pages — adding cross-references, flagging contradictions, updating entity profiles.

The query loop compounds this further. When you ask a question and get a synthesized answer that represents new knowledge, that answer gets filed back as a new wiki page. The slug is derived from the question itself. Tomorrow's queries benefit from today's synthesis.

And there's a maintenance loop most people miss: the lint workflow. Periodically, the LLM audits the entire wiki — finding orphan pages with no incoming links, flagging stale claims, identifying concepts mentioned but never given their own page. The wiki stays healthy because the machine does the maintenance humans always abandon.

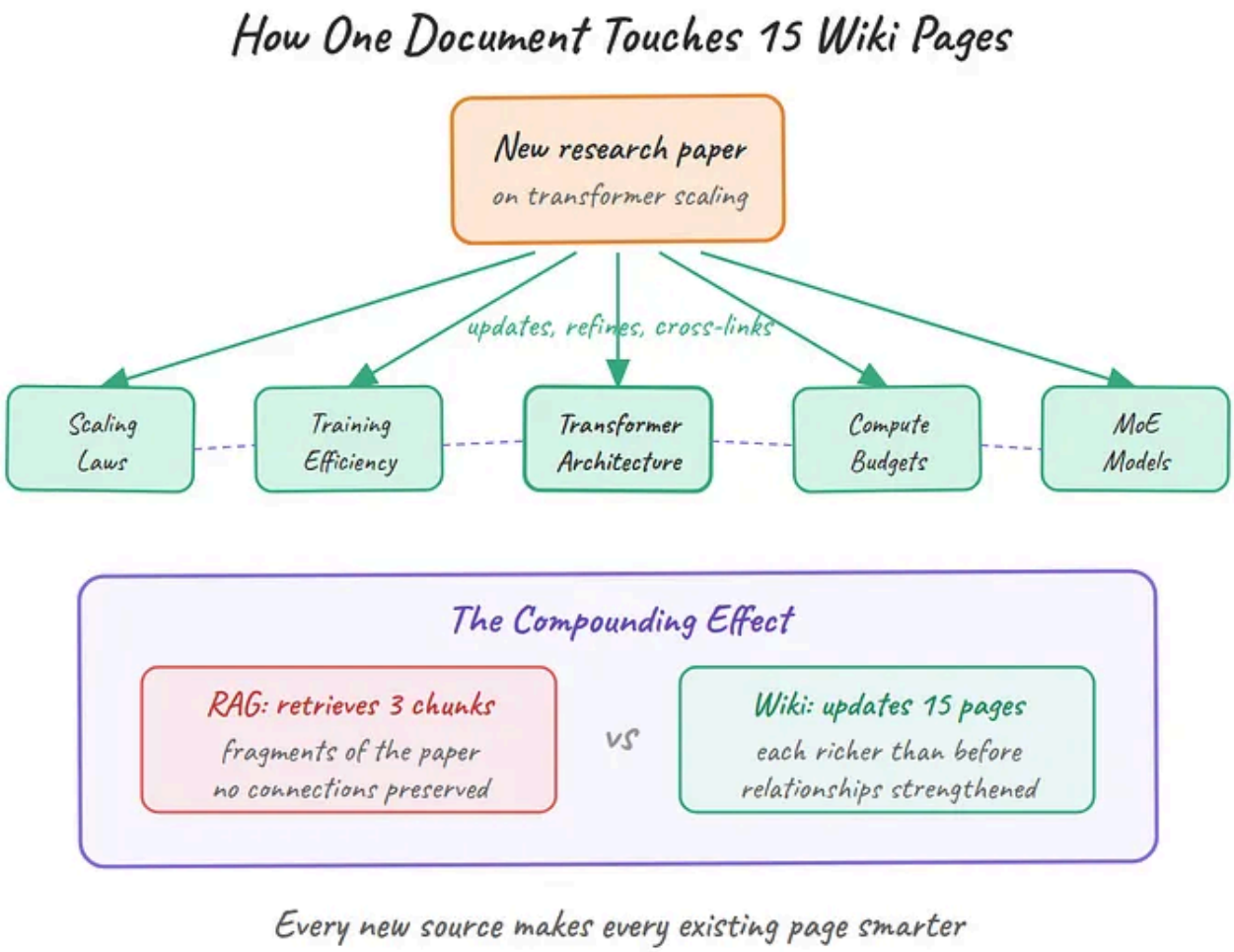

Diagram by Author: How one document touches 15 pages

Where it breaks. Scale is the hard ceiling. The wiki uses markdown files navigated by BM25 or grep. That works beautifully at 100 sources generating a few hundred

wiki pages. At 10,000 sources, the navigation breaks down. At 100,000, it's unusable without adding a retrieval layer on top — which starts to look like RAG again.

There's also the upfront compute cost. Every ingest requires the LLM to read the new source, read the relevant existing wiki pages, and rewrite all affected content. That's significantly more expensive per document than a RAG embedding. For a personal research wiki, the cost is manageable. For an enterprise knowledge base, it's a budget conversation.

And the wiki is passive. It compiles knowledge beautifully, but it doesn't act on it. It won't notice that a deadline mentioned in three sources has passed. It won't trigger a notification when a new document contradicts a standing policy. It knows — but it doesn't do.

The enterprise reality. This architecture works best for researchers, analysts, and small teams building deep expertise in a specific domain. A team tracking regulatory changes across 200 source documents? Excellent fit. A company trying to make 500,000 Confluence pages queryable? Wrong tool.

The data governance story is mixed. The wiki creates derived copies of your source material — summaries, cross-references, synthesis pages. In some regulatory environments, those derived artifacts are themselves subject to retention and audit requirements. That's not a dealbreaker, but it's a conversation your legal team needs to have.

Verdict: Use the LLM Wiki when your sources number in the hundreds (not thousands), your knowledge should compound over time, and you want synthesis — not just retrieval. Don't use it when you need real-time freshness, massive scale, or autonomous action.

Architecture 3: Fat Skills — The Operator

Fat skills win at autonomy and lose at accessibility. If you want knowledge that doesn't just sit there but triggers actions, runs on schedules, and compounds its own capabilities over time, this is the architecture that operates. But it requires real engineering investment — and right now, it's a one-person show.

Garry Tan's GBrain introduces an idea that neither RAG nor the LLM Wiki attempts: knowledge that acts. Not just "what do we know?" but "what should we do about it, and when?"

Important context: *GBrain wasn't designed as an enterprise product. It was built to run on personal AI agents — specifically OpenClaw, Hermes, and Claude Code*. The README is blunt: "Y*our AI agent is smart but forgetful. GBrain gives it a brain.*" This is infrastructure built by the Y Combinator CEO to run his own agents, then open-sourced. That origin shapes everything about the architecture — it optimizes for one power user's workflows, not organizational deployment.

The thin harness, fat skills architecture. GBrain inverts the typical agent design. Most frameworks build a thick runtime with dozens of tool definitions consuming half the context window. GBrain keeps the harness to roughly 200 lines of code — just enough to manage model execution, read/write files, and enforce safety.

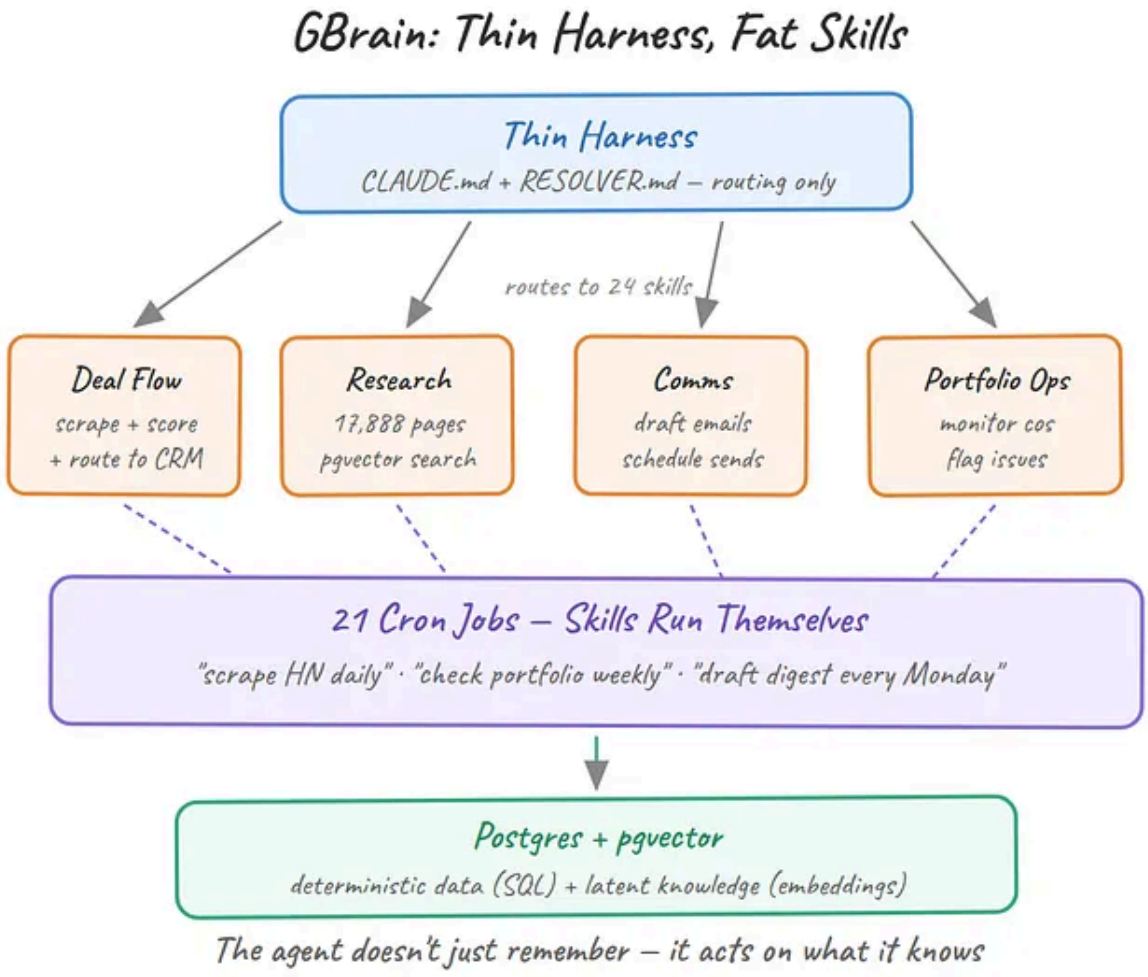

Diagram by Author: Fat skills three-layer architecture

All the intelligence lives in the skills. Each skill is a fat markdown document — not a prompt template, but an entire workflow: when to fire, what to check, how to chain with other skills, what quality bar to enforce. The agent reads the skill file and executes it.

The resolver as routing table. GBrain's RESOLVER.md is a dispatcher that routes user intent to skills across six categories: always-on skills, brain operations, content ingestion, thinking skills, operational tasks, and setup. But here's the insight: the skill descriptions themselves function as the resolver. The model reads the descriptions and matches intent automatically. No explicit routing code needed.

Garry Tan's latest iteration pushes this further: "Fewer fatter skills makes the resolver shorter, which itself is less context bloat. Short resolvers are better than long ones." The trend is toward fewer, more comprehensive skills with branching parameters rather than a library of narrow ones.

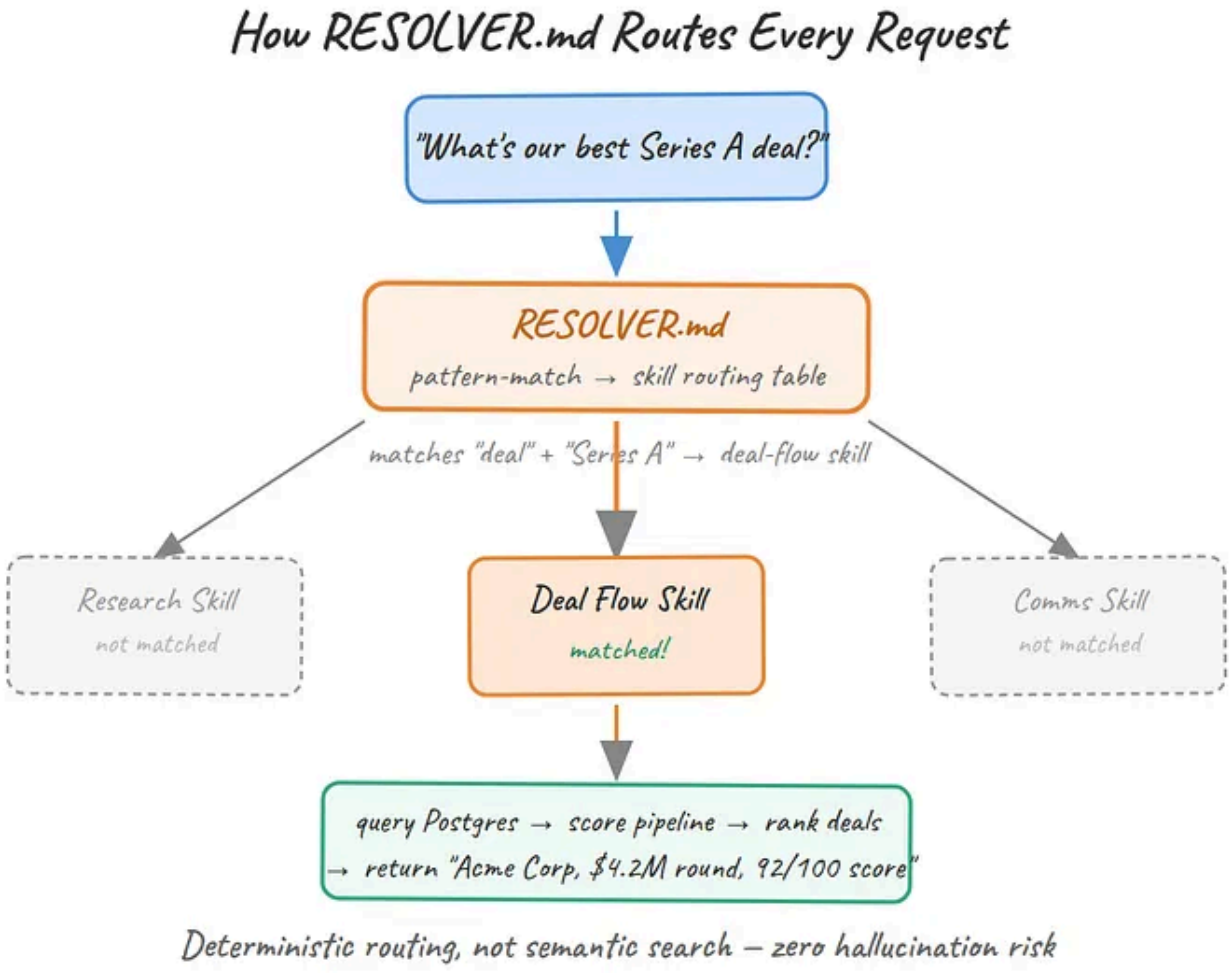

Diagram by Author: Resolver routing flow

What a fat skill actually looks like. Here's the frontmatter from GBrain's `enrich` skill — the one that builds and maintains person and company dossiers:

```yaml
name: enrich
version: 1.0.0
description: |
  Enrich brain pages with tiered enrichment protocol.
  Creates and updates person/company pages with compiled
  truth, timeline, and cross-links.
triggers:
  - "enrich"
  - "create person page"
  - "update company page"
  - "who is this person"
tools:
  - get_page
  - put_page
  - search
  - add_link
  - add_timeline_entry
mutating: true
writes_to:
  - people/
  - companies/
```

That's not a prompt template. It's a contract. The skill declares its triggers, its tools, what it writes to, and whether it mutates brain state. Below this frontmatter sits a seven-step enrichment protocol that runs across three tiers — full research for inner-circle contacts (all APIs, deep web search), moderate effort for industry figures (web + social + brain cross-reference), and light touch for entities worth tracking but not critical. Every claim requires inline `[Source: ...]` citations following a strict precedent hierarchy: user statements rank highest, then compiled truth, then timeline entries, then external APIs.

The skill's philosophy line captures the whole approach: "Intelligence dossiers, not LinkedIn scrapes." The system prioritizes texture — beliefs, current projects, motivations, trajectory — over baseline facts you could get from a Google search.

The always-on layer. Some skills don't wait for triggers at all. GBrain's `signal-detector` fires on every inbound message, running as a cheap sub-agent in parallel with the main response. It captures two things: original ideas (preserved in the

user's exact phrasing, never paraphrased) and entity mentions (people, companies, concepts). Every entity it detects gets cross-linked to existing brain pages or spawns a new one. The operating principle: "An unlinked mention is a broken brain."

This is what separates fat skills from function-calling. A function call is stateless — it runs, returns, and forgets. A fat skill maintains state, enforces quality standards, chains with other skills, and leaves the brain richer after every execution.

Cron: skills that run themselves. The cron scheduler turns skills into autonomous agents. Each job is deliberately thin — the prompt is literally "Read `skills/{name}/SKILL.md` and run it." All the intelligence stays in the skill file, not the scheduler. Jobs run on 5-minute staggered slots to prevent collisions, respect quiet hours (11 PM–8 AM by default), and enforce idempotency — running the same job twice produces identical results with no duplicate outputs. Results get filed to `reports/{job-name}/{YYYY-MM-DD-HHMM}.md` for audit trails.

A typical cron lineup might include: scrape Hacker News every six hours and file new signals, enrich newly mentioned entities daily, check portfolio company metrics weekly, and draft a digest every Monday morning. The agent works while you sleep.

The deterministic split. A skill can call deterministic code — SQL queries, API calls, file operations — for tasks that shouldn't be left to LLM judgment. GBrain separates latent work (reading, synthesis, pattern recognition) from deterministic work (database writes, calculations, reproducible outputs). Mixing them is how agents hallucinate.

Where it breaks. The engineering investment is substantial. GBrain's 24 skills are fully tested with end-to-end tests, evaluations, and unit tests. That's not a weekend project. It's a codebase.

It's also deeply personal. GBrain is built around Garry Tan's specific workflows — his people, his companies, his publishing schedule. The architecture is transferable, but the implementation isn't drag-and-drop. You can't npm install someone else's brain.

And the scale question is different from RAG or Wiki. GBrain's brain repo spans 17,888 pages — impressive for a personal system, small for an enterprise. The Postgres + pgvector backend could theoretically scale further, but the skill architecture assumes a single operator who understands the full system.

The enterprise reality. This architecture maps well to a specific enterprise pattern: the power user who serves as a force multiplier for their team. A senior analyst running autonomous research workflows. A product lead who needs competitive intelligence updated daily without asking for it. An engineering manager whose agent monitors deployment health and escalates autonomously.

But it doesn't map to the typical enterprise deployment of "give everyone access to a knowledge assistant." The engineering overhead per user is too high, and the skill authoring requires a level of system understanding that most knowledge workers don't have. At least, not yet.

Verdict: Use fat skills when your knowledge needs to trigger autonomous actions, when you're willing to invest in engineering the skill layer, and when one power user can define the workflows for a team. Don't use it when you need broad organizational access or when your team can't maintain the skill codebase.

The Comparison: Same Problem, Three Trade-offs

Here's what the "RAG is dead" discourse misses: these architectures aren't competing. They're solving different versions of the same problem. Picking between them is a design decision, not a loyalty test.

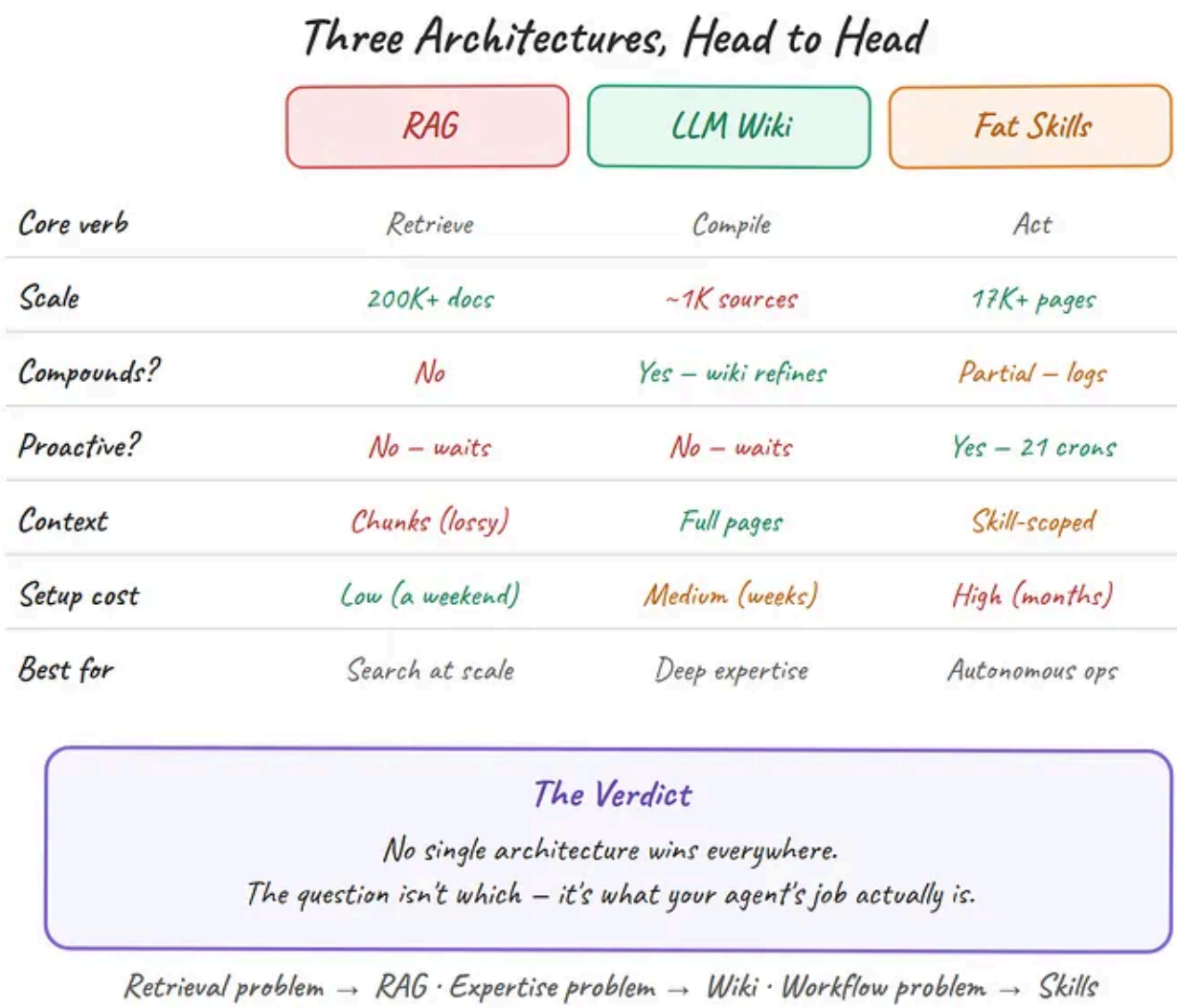

Diagram by Author: Three-way comparison matrix

The decision starts with one question: what is your agent's job?

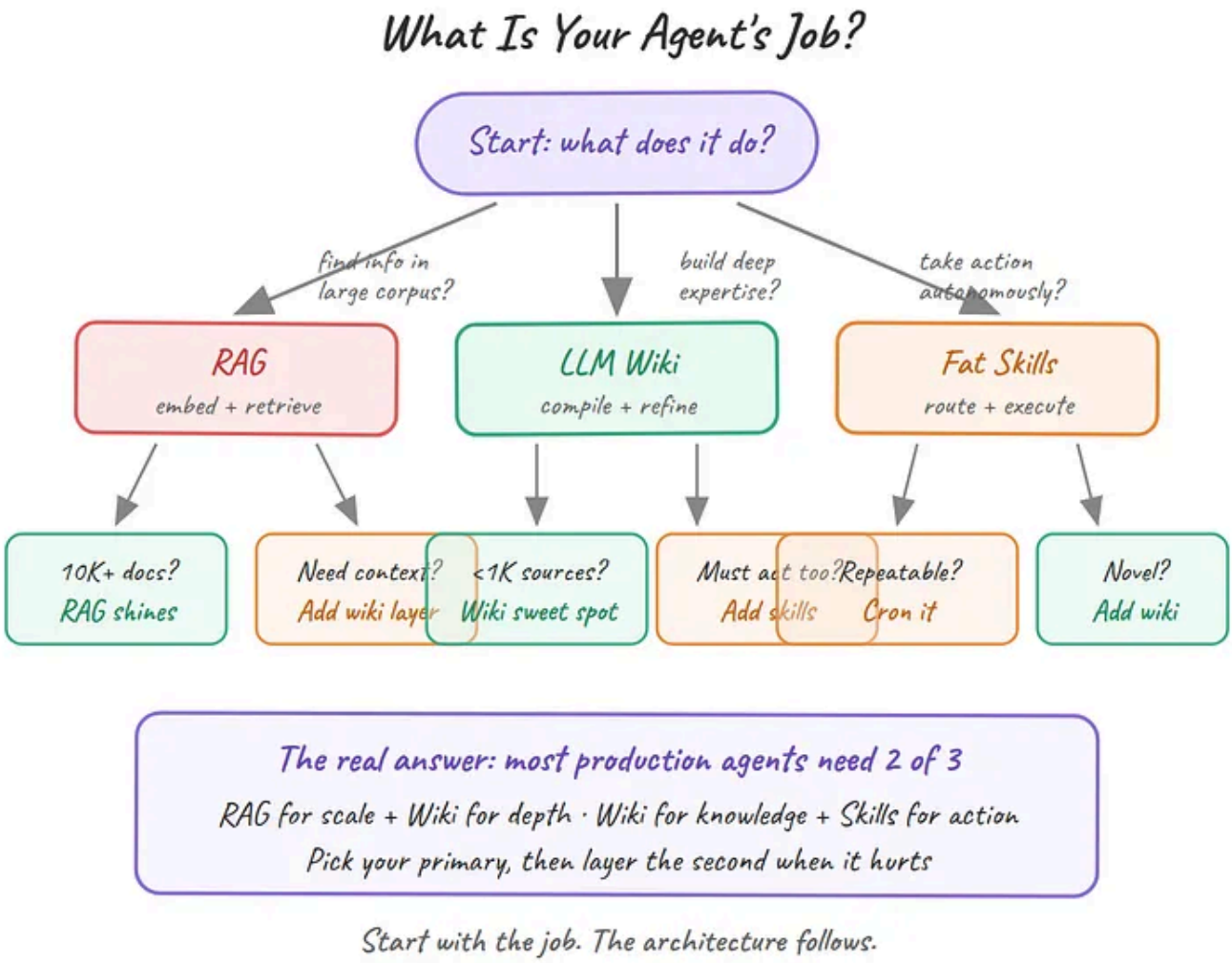

Diagram by Author: Decision tree — What is your agent's job?

Your agent retrieves answers from a large corpus. You have thousands of documents. They change regularly. Users ask questions and need answers fast. Your priority is shipping something production-ready with a known cost model.

Use RAG. It's not glamorous, but it's the architecture that scales to the corpus sizes most organizations actually have. Pair it with a reranker and you'll cover 80% of knowledge assistant use cases.

Your agent builds expertise that should compound over time. You have hundreds of sources — research papers, regulatory filings, competitor analyses, technical specs. The value isn't in any single document but in the connections between them. You want your agent to get smarter the longer you use it.

Use the LLM Wiki. Feed it your sources, let it build the cross-references, and query the compiled wiki instead of raw chunks. Your hundredth query will be significantly better than your first — something RAG can never promise.

Your agent needs to act on what it knows without being asked. You don't just want answers. You want your agent to monitor, flag, enrich, and execute. You want it running while you sleep, filing insights back into the system, triggering workflows when conditions change.

Use fat skills. But budget for the engineering. This isn't a weekend hack — it's a codebase with tests, evaluations, and maintenance overhead. The payoff is an agent that operates, not just responds.

The hybrid reality. Production systems won't stay in a single lane. The most capable architectures will combine all three: RAG for the retrieval layer (finding relevant content at scale), Wiki for the synthesis layer (compiling retrieved content into persistent knowledge), and skills for the action layer (operationalizing that knowledge into autonomous workflows).

Claude Code already hints at this convergence. CLAUDE.md files function as a mini-wiki (persistent context that compounds across sessions). Auto-memory learns from interactions (compounding). Skills trigger workflows (action). It's not a deliberate implementation of all three patterns — but the same pressures produced the same solutions.

Where This Is Heading

The three-way split won't last. The same way databases evolved from "pick SQL or NoSQL" to hybrid systems that handle both, knowledge architectures are converging.

The early signs are already visible. Karpathy's LLM Wiki v2 community extensions are adding retrieval layers on top of compiled wikis — compounding meets scale. GBrain's skills already query a Postgres + pgvector backend — action meets retrieval. And enterprise platforms like Neo4j are building knowledge layers that combine graph databases, vector search, and semantic reasoning in a single access point.

The question for 2026 isn't "which architecture wins." It's how quickly the boundaries between retrieve, compile, and act dissolve into a single knowledge operating system that does all three.

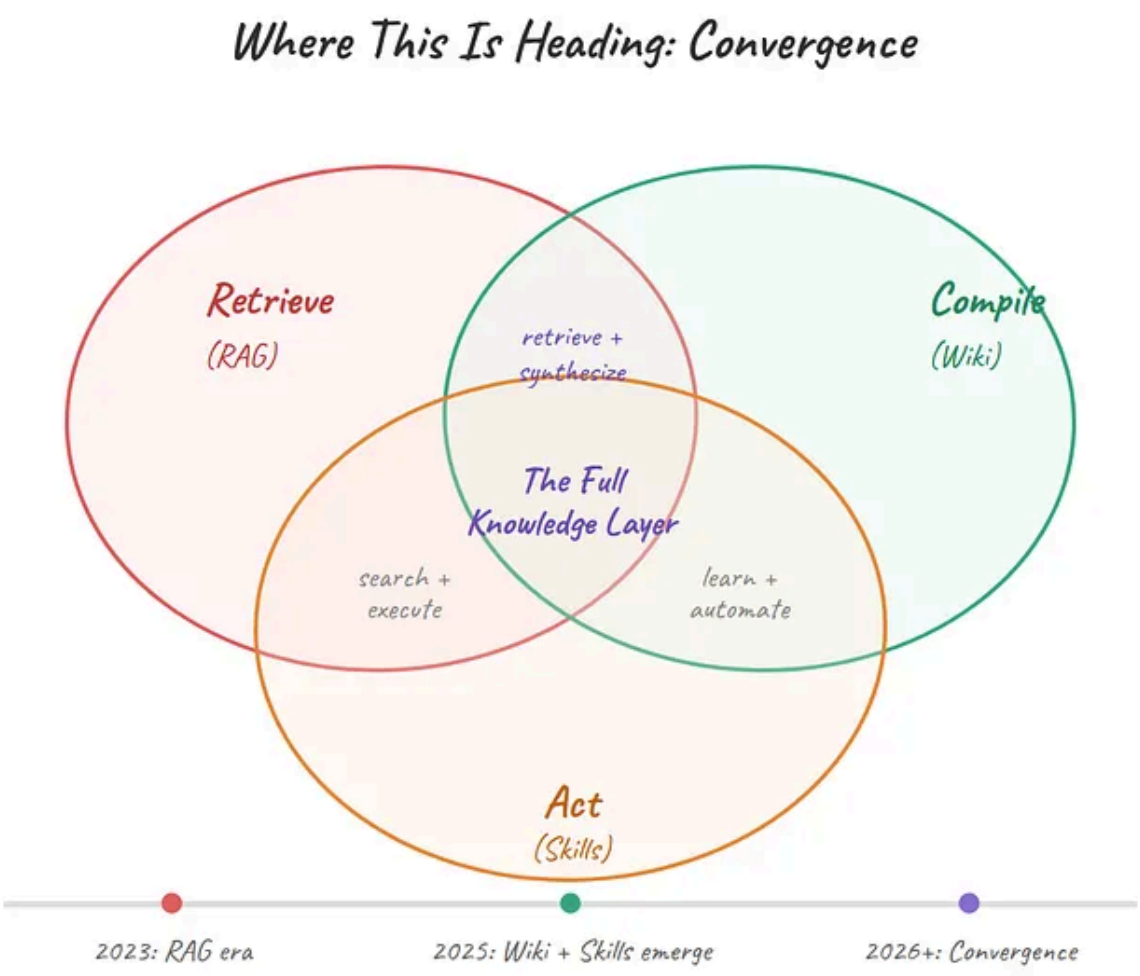

Diagram by Author: Convergence — Retrieve, Compile, Act overlapping

If you want to start exploring today:

- **RAG:** LangChain's RAG tutorial is still the fastest path to a working prototype
- **LLM Wiki:** Karpathy's gist is 200 lines of schema that you can run against Claude or GPT today
- **Fat Skills:** GBrain's repo is open-source — read RESOLVER.md and THIN_HARNESS_FAT_SKILLS.md before you read any code

The knowledge layer is the part of the agent stack that nobody talks about until it breaks. Now you know what your options are.

OpenAI Quietly Told You to Throw Away Your Prompt Stack

And Anthropic said the same thing. The 3 eras of prompting — and what the smartest models actually want from you.

ai.gopubby.com

OpenAI Symphony vs Claude Managed Agents vs CrewAI: Which Agent Orchestration Pattern Wins

Three architectures benchmarked. $617K in annual cost difference. One Pareto-optimal winner.

ai.gopubby.com

The 4 Lines Every CLAUDE.md Needs

What Karpathy diagnosed, what 60,000 developers bookmarked, and why behavioral constraints beat feature checklists

levelup.gitconnected.com

Harness Engineering: What Every AI Engineer Needs to Know in 2026

Three camps, three architectures — and what Opus 4.7 just proved about all of them

ai.gopubby.com

Before you go! 🦸

If you liked my story and you want to support me:

1. Throw some Medium love 💖(claps, comments and highlights), your support means the world to me.👏
2. Follow me on Medium and subscribe to get my latest article

About - Yanli Liu - Medium

Read writing from Yanli Liu on Medium. Daytime finance practitioner based in Luxembourg, seasoned coder, and passionate...

medium.com

Programming Technology Artificial Intelligence

Follow

Published in AI Advances

79K followers · Last published 5 hours ago

Democratizing access to artificial intelligence

Follow

Written by Yanli Liu

5.8K followers · 41 following

Daytime finance practitioner based in Luxembourg, seasoned coder, and passionate about anything AI-related!

Responses (9)

Datachamp

What are your thoughts?

AIPPV
Apr 30

Thank you very much.

5 Reply

Chesz AI
May 5

This is a sharp framing of a problem most teams are already running into: memory architecture is becoming the real bottleneck in agent performance.

RAG works well for retrieval, but it's fundamentally stateless; every query starts from scratch. What... more

3 Reply

MD . Afzal khan he
May 1

it is too Easy and understanding Thanks a lot for deep explanation

8 Reply

See all responses